Recursividad y Backtracking

ICPC camp Potosí 2026

Recursividad

La recursividad es un concepto en programación en el que una función se llama a sí misma directa o indirectamente para resolver un problema.

Características:

- Una función recursiva tiene al menos un caso base que define la condición de salida.
- Cada llamada recursiva debe acercarse al caso base para evitar un bucle infinito.

Recursividad: Factorial

```
int factorial(int n) {
    if (n <= 1) {
        return 1; // caso base
    }
    return n * factorial(n - 1); // llamada recursiva
}
```

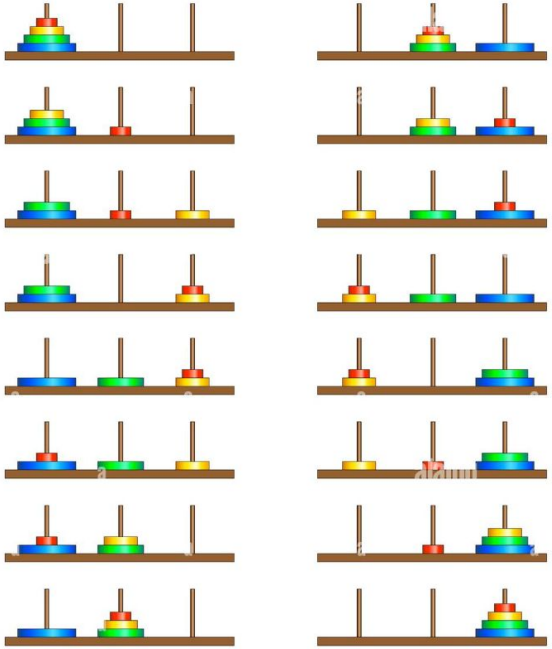
Recursividad: más ejemplos

Tengo una caja de tamaño 1*N:

- es posible llenarla con bloques de 1*2?
- es posible llenarla con bloques de 1*3?
- es posible llenarla con bloques de 1*K?
- de cuántas formas se la puede llenar combinando bloques de 1*1 y 1*2?
- de cuántas formas se la puede llenar combinando bloques de 1*1 y 1*K?
- de cuántas formas se la puede llenar combinando bloques de 1*K y 1*L?

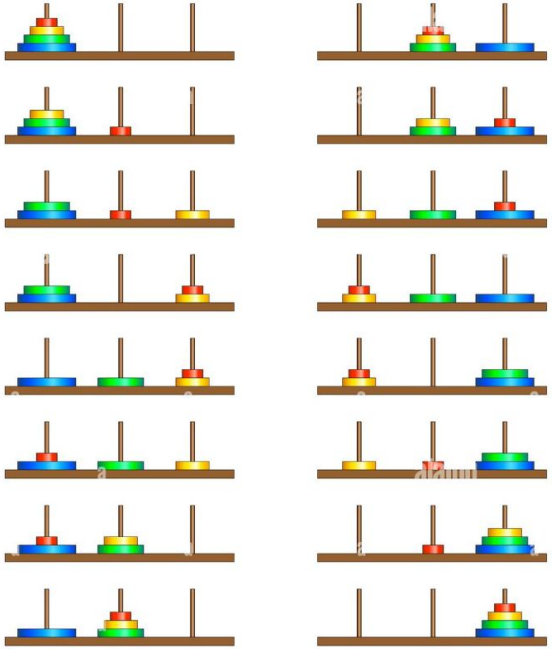
Torres de Hanoi

- El problema consiste en una torre con discos de diferentes tamaños colocados en una varilla en orden descendente de tamaño.
- El objetivo es mover todos los discos a otra varilla, respetando las siguientes reglas:
  - Solo se puede mover un disco a la vez.
  - Un disco más grande nunca puede estar sobre uno más pequeño.



Torres de Hanoi: Solución

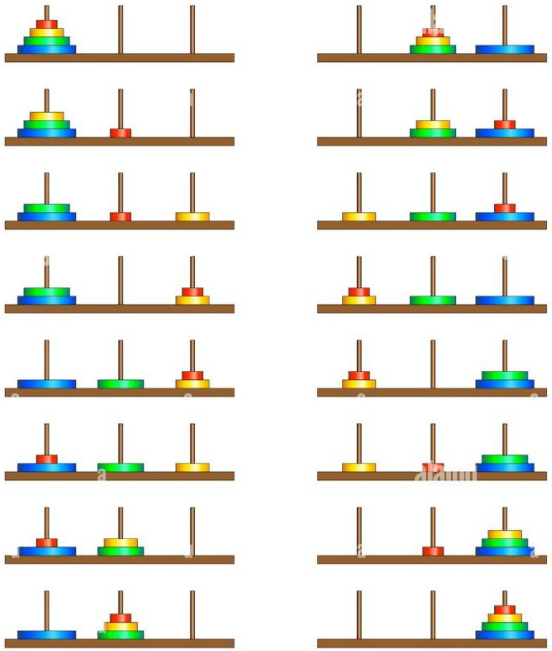
- La solución se basa en el principio de dividir y conquistar.
- Para mover n discos de la varilla A a la varilla C, se puede hacer lo siguiente:
  a. Mover los n-1 discos superiores de A a una varilla auxiliar B, utilizando C como varilla auxiliar.
  b. Mover el disco más grande de A a C.
  c. Mover los n-1 discos de B a C, utilizando A como varilla auxiliar.



Torres de Hanoi: Solución

Para 3 discos:

1. Mover disco 1 de A a C
2. Mover disco 2 de A a B
3. Mover disco 1 de C a B
4. Mover disco 3 de A a C
5. Mover disco 1 de B a A
6. Mover disco 2 de B a C
7. Mover disco 1 de A a C



Torres de Hanoi: Código

```
void hanoi(int n, int origen, int destino, int auxiliar) {
    if (n == 1) {
        cout <<"Mover disco 1 de "<<origen<<" a "<<destino<<'\n';
        return;
    }
    hanoi(n - 1, origen, auxiliar, destino);
    cout <<"Mover disco "<<n<<" de "<<origen<<" a "<<destino<<'\n';
    hanoi(n - 1, auxiliar, destino, origen);
}
```

Generar Subsets

Clásico ejemplo de “tomar o no tomar”

{ }

{ 3 }

{ 2 }

{ 2 3 }

{ 1 }

{ 1 3 }

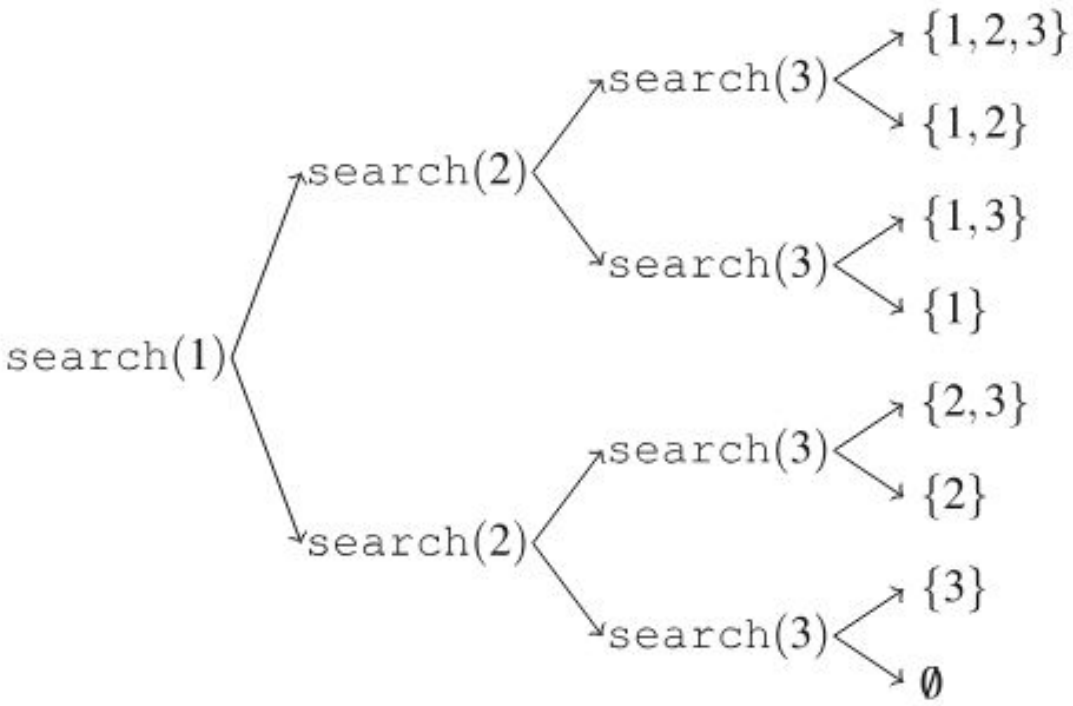
{ 1 2 }

{ 1 2 3 }

Generar Subsets

```
vector<int> subset;
void search(int k) {
    if (k == n+1) {
        // process subset
    } else {
        // include k in the subset
        subset.push_back(k);
        search(k+1);
        subset.pop_back();
        // don't include k in the subset
        search(k+1);
    }
}
```

Generar Subsets



Generar Permutaciones

Parecido a subsets, pero se toma un orden lexicográfico y no existe el “no tomar”.

{ 1 2 3 }

{ 1 3 2 }

{ 2 1 3 }

{ 2 3 1 }

{ 3 2 1 }

{ 3 1 2 }

Generar Permutaciones

```
vector<int> permutation;
bool chosen[n+1];
void search() {
    if (permutation.size() == n) {
        // process permutation
    } else {
        for (int i = 1; i <= n; i++) {
            if (chosen[i]) continue;
            chosen[i] = true;
            permutation.push_back(i);
            search();
            chosen[i] = false;
            permutation.pop_back();
        }
    }
}
```

Generar Permutaciones

Usando el next_permutation:

```
for (int i = 1; i <= n; i++) {
perm.push_back(i);
}
do {
// process permutation
} while (next_permutation(perm.begin(), perm.end()));
```

Backtracking

- El backtracking es una técnica de búsqueda exhaustiva que consiste en probar todas las posibles soluciones a un problema de manera sistemática.
- Características:
  - Utiliza la recursividad para probar cada opción.
  - Retrocede (backtracks) cuando encuentra una solución incorrecta

8 queens

- Colocar 8 reinas en un tablero de ajedrez de 8x8 sin que se amenacen entre sí.
- Las reinas pueden moverse horizontalmente, verticalmente y en diagonal
- No puede haber dos reinas en la misma fila, columna o diagonal.

8 queens

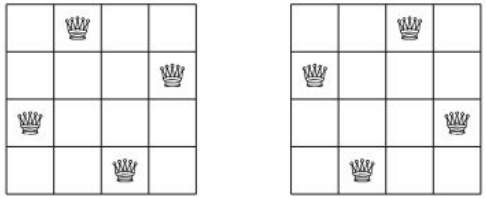


Fig. 2.2 The possible ways to place 4 queens on a 4 × 4 chessboard

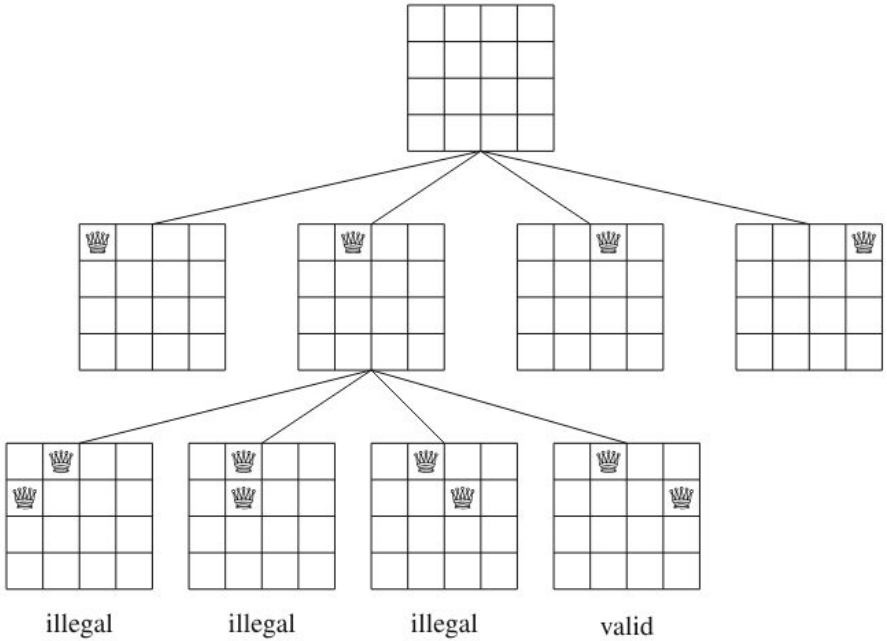


Fig. 2.3 Partial solutions to the queen problem using backtracking

8 Queens: Código

```
void search(int y) {
    if (y == n) {
        count++;
        return;
    }
    for (int x = 0; x < n; x++) {
        if (col[x] || diag1[x+y] || diag2[x-y+n-1]) continue;
        col[x] = diag1[x+y] = diag2[x-y+n-1] = 1;
        search(y+1);
        col[x] = diag1[x+y] = diag2[x-y+n-1] = 0;
    }
}
```

8 queens

De hecho, nadie conoce una forma eficiente de contar el número de combinaciones de reinas
para valores mayores de n. Actualmente, el mayor valor de n para el que se conoce el resultado es
27: en este caso hay 234907967154122528 combinaciones. Esto fue descubierto
en 2016 por un grupo de investigadores que utilizó un clúster de ordenadores para calcular el
resultado

Diferencias entre Backtracking y Búsqueda Completa

Definición:

Backtracking: Es una técnica para encontrar soluciones recursivamente probando todas las posibles opciones y retrocediendo cuando se llega a una solución incorrecta.

Búsqueda completa: Es un enfoque general para resolver problemas que implica enumerar sistemáticamente todas las posibles soluciones.

Diferencias entre Backtracking y Búsqueda Completa

Objetivo:

Backtracking: Encuentra una solución para un problema específico.

Búsqueda completa: Encuentra todas las soluciones posibles para un problema.

Diferencias entre Backtracking y Búsqueda Completa

Estructura:

Backtracking: Utiliza recursión para probar todas las opciones y retroceder cuando sea necesario.

Búsqueda completa: Puede utilizar diferentes enfoques según el problema, como fuerza bruta, backtracking, divide y vencerás, entre otros.

Diferencias entre Backtracking y Búsqueda Completa

Complejidad:

Backtracking: La complejidad puede variar según el problema y cómo se implemente la técnica de backtracking.

Búsqueda completa: Tiene una complejidad alta ya que considera todas las posibles soluciones.

Diferencias entre Backtracking y Búsqueda Completa

Ejemplos:

Backtracking: Problema de las 8 reinas, sudoku, resolver laberintos.

Búsqueda completa: Problema del vendedor viajero, encontrar todas las permutaciones de un conjunto.

P vs NP y su relación con el backtracking

- **P vs NP** es uno de los problemas más importantes en la teoría de la computación.
- **P:** Conjunto de problemas que pueden ser resueltos de manera eficiente (en tiempo polinómico) por una computadora.
- **NP**: Conjunto de problemas cuyas soluciones pueden ser verificadas de manera eficiente, pero no necesariamente encontradas de manera eficiente.
- Si un problema **NP** puede ser resuelto de manera eficiente, entonces **P = NP**.
- Backtracking es un enfoque común para resolver problemas NP, pero no garantiza soluciones eficientes.